Examen: Conversión de Aplicación de Recetario Estrafalario a React

Introducción

El objetivo de este proyecto es tomar una aplicación existente de visualización de recetas de cocina, desarrollada en **HTML, CSS y JavaScript**, y convertirla a **React** utilizando **Vite**. Deberán seguir las mejores prácticas y patrones de diseño de React, aprovechando los hooks y herramientas clave, tales como `useState`, `useEffect`, `react-router`, `useNavigate`, `useContext`, y **Custom Hooks**.

Servidor API

IMPORTANTE: Este proyecto utiliza un servidor API mock que debe estar corriendo para que la aplicación funcione correctamente.

Iniciar el Servidor

El servidor se encuentra en la carpeta `server/`. Para iniciarlo:

```
# Desde la raíz del proyecto
cd server
npm install  # Solo la primera vez
node mockServer.js
```

El servidor correrá en `http://localhost:3000` y la API estará disponible en `http://localhost:3000/api/`.

Endpoints Disponibles

- `GET /api/recetas` - Obtiene todas las recetas (con filtros opcionales: `?categoria=Chicken&area=Indian&buscar=chicken`)
- `GET /api/recetas/:id` - Obtiene una receta específica por ID
- `GET /api/categorias` - Obtiene todas las categorías disponibles
- `GET /api/areas` - Obtiene todas las áreas/países disponibles
- `GET /api/recetas/categoria/:categoria` - Obtiene recetas por categoría
- `GET /api/recetas/area/:area` - Obtiene recetas por área

Requerimientos Técnicos

La conversión debe implementar lo siguiente:

- **useState**: Para manejar el estado de las recetas, filtros, búsqueda y otros datos relevantes.
- **useEffect**: Para manejar la carga inicial de datos desde la API y la actualización del componente.
- **react-router**: Para manejar la navegación entre la página principal, recetas recientes y la página de detalles de la receta.
- **useNavigate**: Para redirigir a los usuarios entre las diferentes páginas (por ejemplo, después de seleccionar una receta).
- **useContext**: Para manejar el estado global de **recetas recientes**. Este estado debe persistir durante la navegación y mostrar las últimas recetas vistas por el usuario (máximo 15 recetas).
- **Custom Hooks**: Para encapsular la lógica de gestión de recetas recientes (agregar, obtener, limpiar).

Pasos para la Conversión

1. Inicializar el Proyecto con Vite

- Crea un nuevo proyecto utilizando **Vite**.
- Organiza la estructura del proyecto en carpetas adecuadas (`components`, `hooks`, `context`, `pages`, etc.).
- **Asegúrate de que el servidor API esté corriendo** antes de iniciar el desarrollo.

Ejemplo de Fetch en React:

En React con Vite, las peticiones a la API se realizan directamente al servidor:

```
// Ejemplo de carga de recetas
useEffect(() => {
  const fetchRecetas = async () => {
    try {
      const response = await fetch('http://localhost:3000/api/recetas');
```

```
      const data = await response.json();
      if (data.success) {
        setRecetas(data.recetas);
      }
    } catch (error) {
      console.error('Error al cargar recetas:', error);
    }
  };

  fetchRecetas();
}, []);

// Ejemplo de carga de receta individual
const fetchReceta = async (id) => {
  try {
    const response = await fetch(`http://localhost:3000/api/recetas/${id}`);
    const data = await response.json();
    if (data.success) {
      return data.receta;
    }
  } catch (error) {
    console.error('Error al cargar receta:', error);
    return null;
  }
};
```

Nota: Todas las peticiones deben realizarse a `http://localhost:3000/api/` seguido del endpoint correspondiente.

2. Crear las Rutas con `react-router`

- Implementa el enrutamiento con **react-router**:
  - Ruta principal (`/`): Donde se mostrarán las recetas disponibles con filtros por categoría, área y búsqueda por nombre.
  - Ruta de recetas recientes (`/recientes`): Donde se mostrarán las últimas recetas vistas por el usuario.
  - Ruta de detalles de la receta (`/receta/:id`): Donde se mostrará la información completa de la receta seleccionada.
  - **Ruta 404 (`*`)**: Para manejar páginas no encontradas. Debe mostrar un mensaje de error temático (con emojis de cocina) y un botón para volver al inicio.

3. Página Principal (Componente `Home`)

- **useState**: Para manejar la lista de recetas, filtros activos (categoría, área, búsqueda) y estado de carga.
- **useEffect**: Para cargar las recetas desde la API cuando cambien los filtros.
- **useNavigate**: Al hacer clic en una receta, navega a la página de detalles usando `useNavigate()` .
- Debe incluir:
  - Filtro por categoría (dropdown)
  - Filtro por área/país (dropdown)
  - Búsqueda por nombre de receta (input de texto)
  - Grid de recetas con cards mostrando imagen, nombre, categoría y área

4. Página de Recetas Recientes (Componente `Recientes`)

- **useContext**: Debe usar el contexto de recetas recientes para obtener la lista de IDs de recetas vistas.
- **useState**: Para manejar las recetas cargadas y estado de carga.
- **useEffect**: Para cargar los detalles de cada receta reciente desde la API.
- Muestra las últimas recetas vistas (máximo 15), ordenadas de más reciente a más antigua.
- Si no hay recetas recientes, mostrar un mensaje amigable.

5. Página de Detalles de la Receta (Componente `RecetaDetail`)

- **useEffect**: Para leer el ID de la receta desde la URL usando `useParams()` y cargar los datos correspondientes desde la API.
- **useState**: Para manejar los detalles de la receta y estado de carga.
- **useContext**: Debe agregar la receta actual a las recetas recientes cuando se visualiza.
- **useNavigate**: Botón para volver al catálogo.

- Muestra la información completa de la receta:
  - Imagen
  - Nombre
  - Categoría y área
  - Lista de ingredientes con sus medidas
  - Instrucciones de preparación
  - Enlace a video de YouTube (si está disponible)
- Incluye un botón para **volver al catálogo**.

6. Página 404 (Componente `NotFound`)

- Crea un componente para manejar rutas no encontradas.
- Debe mostrar:
  - Un emoji de cocina (🍳).
  - Un mensaje de error **404 - Página no encontrada**.
  - Una descripción amigable del error relacionada con la temática (ejemplo: "Parece que esta receta se perdió en la cocina 🍳").
  - Un botón para **volver al inicio** usando `useNavigate()`.
- Mantén la estética consistente con el resto de la aplicación.

7. Sistema de Recetas Recientes con Context API

- **useContext**: Crea un contexto (`RecetasRecientesContext`) para manejar el estado global de recetas recientes.
  - El estado debe almacenar un array de IDs de recetas (máximo 15).
  - Cuando se visualiza una receta, debe agregarse al inicio del array.
  - Si la receta ya existe en el array, debe moverse al inicio.
  - El estado debe persistir en `localStorage` para mantener las recetas recientes entre sesiones.
  - Debe estar disponible en todas las páginas.

8. Implementación de Custom Hooks

- **useRecetasRecientes**: Un custom hook para manejar la lógica de recetas recientes.
  - Funciones para agregar una receta a recientes.
  - Función para obtener la lista de IDs de recetas recientes.
  - Función para limpiar el historial de recetas recientes.
  - Manejo de persistencia en `localStorage`.

Requerimientos Funcionales

1. **Visualización de recetas**: La lista de recetas debe cargarse dinámicamente desde la API (`http://localhost:3000/api/recetas`), con soporte para filtros por categoría, área y búsqueda por nombre.

2. **Filtros y búsqueda**: Los filtros deben actualizar la lista de recetas en tiempo real. La búsqueda debe tener un debounce para evitar demasiadas peticiones.

3. **Recetas recientes**: El sistema debe rastrear automáticamente las recetas vistas y mostrarlas en la página de recientes. El estado debe persistir en `localStorage`.

4. **Navegación**: Implementar navegación entre la página principal, recetas recientes, detalles de receta y página 404.

5. **Página 404**: Debe mostrar un mensaje de error amigable y temático cuando se acceda a una ruta inexistente, con opción de volver al inicio.

6. **Hooks personalizados**: Se deben implementar hooks personalizados para la lógica de recetas recientes.

7. **Responsive**: La aplicación debe verse correctamente en dispositivos móviles y desktop.

8. **Manejo de errores**: Debe manejar correctamente los errores de red y mostrar mensajes apropiados al usuario.

Estructura Sugerida del Proyecto React

```
recetario-react/
├── server/
│   ├── mockServer.js
│   └── README.md
```

```
├── src/
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── RecetaCard.jsx
│   │   ├── Filtros.jsx
│   │   ├── Loading.jsx
│   │   └── ErrorMessage.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Recientes.jsx
│   │   ├── RecetaDetail.jsx
│   │   └── NotFound.jsx
│   ├── context/
│   │   └── RecetasRecientesContext.jsx
│   ├── hooks/
│   │   └── useRecetasRecientes.js
│   ├── services/
│   │   └── api.js
│   ├── App.jsx
│   └── main.jsx
└── package.json
```

Evaluación

1. **Funcionalidad**: La aplicación debe funcionar correctamente en React y comunicarse con la API.
2. **Uso de Hooks**: Se evaluará el uso adecuado de `useState`, `useEffect`, `useNavigate`, `useParams`, `useContext`, y custom hooks.
3. **Modularidad**: El código debe estar bien organizado y estructurado en componentes reutilizables.
4. **Estado Global**: El estado de recetas recientes debe manejarse correctamente con `useContext` y reflejarse en todas las páginas.
5. **Manejo de errores**: Implementación correcta de la página 404 y manejo de errores en las peticiones a la API.
6. **Carga de datos**: Correcta implementación del fetch a la API del servidor.
7. **Persistencia**: Las recetas recientes deben persistir correctamente en `localStorage`.
8. **Diseño**: Mantener la estética consistente y funcionalidad responsive.

Entrega

Los alumnos deberán subir el proyecto a un repositorio de **GitHub**, con instrucciones claras para ejecutarlo (incluyendo cómo iniciar el servidor API), y enviar el enlace antes de la fecha límite.

Rúbrica de Evaluación

Puntaje Total: 100 puntos

1. React Router - Navegación (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Implementación de rutas	Todas las rutas funcionan correctamente: `/` (Home), `/recientes` (Recientes), `/receta/:id` (Detalles), `*` (404). Navegación fluida con `useNavigate`	Rutas principales funcionan, puede faltar 404 o tener errores menores en navegación	Rutas implementadas pero con errores significativos en la navegación	No implementa react-router o rutas no funcionales

Puntos clave a evaluar:

- ✅ Instalación de `react-router-dom`
- ✅ Configuración de `BrowserRouter`
- ✅ Ruta principal `/` funcional
- ✅ Ruta `/recientes` funcional
- ✅ Ruta dinámica `/receta/:id` funcional
- ✅ Ruta 404 `*` implementada

- ✅ Uso correcto de `useNavigate()` para navegación programática
- ✅ Uso correcto de `useParams()` para obtener ID de la URL

---

2. useState - Manejo de Estado Local (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Uso de useState	`useState` usado correctamente en múltiples componentes: lista de recetas, filtros, búsqueda, detalles de receta, estados de carga	`useState` implementado en componentes principales con algunos errores menores	`useState` usado pero de forma incorrecta o incompleta	No usa `useState` o su uso es completamente erróneo

Puntos clave a evaluar:

- ✅ Estado para lista de recetas en Home
- ✅ Estado para filtros (categoría, área, búsqueda)
- ✅ Estado para detalles de receta en RecetaDetail
- ✅ Estados de carga (loading) implementados
- ✅ Estados de error implementados
- ✅ Manejo correcto de la actualización del estado

---

3. useEffect - Efectos y Carga de Datos (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Implementación de useEffect	`useEffect` usado correctamente para cargar datos desde la API. Manejo adecuado de dependencias, carga cuando cambian filtros, cleanup cuando necesario	`useEffect` implementado para carga de datos con errores menores en dependencias	`useEffect` usado pero con problemas significativos (bucles infinitos, dependencias incorrectas)	No usa `useEffect` o su uso genera errores críticos

Puntos clave a evaluar:

- ✅ Carga de recetas desde la API en Home
- ✅ Carga de receta individual en RecetaDetail usando `useParams()`
- ✅ Carga de recetas recientes en componente Recientes
- ✅ Array de dependencias correcto para filtros
- ✅ Manejo de errores en peticiones fetch
- ✅ Estados de carga mientras se obtienen datos

---

4. useContext - Estado Global de Recetas Recientes (15 puntos)

Criterio	Excelente (13-15)	Bueno (10-12)	Regular (6-9)	Insuficiente (0-5)
Context API implementado	`RecetasRecientesContext` creado y usado correctamente. Estado de recetas recientes disponible en toda la app. Agregar receta funciona perfectamente. Persistencia en localStorage implementada. Máximo 15 recetas, sin duplicados	Context implementado, funciona pero con errores menores en la sincronización del estado o persistencia	Context creado pero con problemas significativos en su implementación o uso	No implementa `useContext` o no funciona

Puntos clave a evaluar:

- ✅ Archivo `RecetasRecientesContext.jsx` creado
- ✅ Provider envolviendo la aplicación
- ✅ Estado de recetas recientes compartido globalmente
- ✅ Funciones para agregar, obtener y limpiar recetas recientes
- ✅ Persistencia en `localStorage`
- ✅ Límite de 15 recetas implementado
- ✅ Si receta ya existe, se mueve al inicio (sin duplicados)

- ✅ Estado persiste durante la navegación

---

5. Custom Hook - useRecetasRecientes (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Custom Hook implementado	`useRecetasRecientes` hook personalizado implementado correctamente. Encapsula lógica de recetas recientes, retorna funciones y estados necesarios	Hook creado y funcional con errores menores	Hook creado pero no encapsula correctamente la lógica o tiene errores	No implementa custom hook

Puntos clave a evaluar:

- ✅ Archivo `useRecetasRecientes.js` en carpeta `hooks/`
- ✅ Lógica de agregar receta encapsulada
- ✅ Lógica de obtener recetas recientes encapsulada
- ✅ Lógica de limpiar historial encapsulada
- ✅ Retorna funciones y estados necesarios
- ✅ Reutilizable y siguiendo convenciones de hooks

---

6. Página Principal - Home (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Componente Home funcional	Muestra todas las recetas correctamente con filtros funcionando, búsqueda con debounce, cards interactivas, navegación a detalles funciona perfectamente, diseño atractivo y responsive	Muestra recetas, navegación funciona con errores menores de UI o filtros	Muestra recetas pero con problemas en navegación, filtros o diseño	No muestra recetas o no funciona

Puntos clave a evaluar:

- ✅ Renderiza lista completa de recetas desde la API
- ✅ Filtro por categoría funcional
- ✅ Filtro por área funcional
- ✅ Búsqueda por nombre funcional con debounce
- ✅ Componentes `RecetaCard` reutilizables
- ✅ Cards muestran información básica (imagen, nombre, categoría, área)
- ✅ Click en receta navega a detalles
- ✅ Diseño responsive
- ✅ Manejo de estado de carga
- ✅ Manejo de errores de red

---

7. Página de Recetas Recientes - Recientes (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Componente Recientes funcional	Muestra correctamente las últimas recetas vistas (máximo 15), ordenadas de más reciente a más antigua. Carga detalles desde la API. Muestra mensaje cuando no hay recetas. Usa el contexto correctamente	Muestra recetas recientes con errores menores (puede no cargar todas o tener problemas de orden)	Muestra recetas pero con problemas significativos o no usa el contexto	No muestra recetas recientes o no funciona

Puntos clave a evaluar:

- ✅ Usa `useContext` para obtener IDs de recetas recientes
- ✅ Carga detalles de cada receta desde la API
- ✅ Muestra máximo 15 recetas

- ✅ Ordenadas de más reciente a más antigua
- ✅ Muestra mensaje amigable cuando no hay recetas
- ✅ Manejo de estado de carga
- ✅ Manejo de errores

---

8. Página de Detalles - RecetaDetail (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Componente RecetaDetail funcional	Muestra todos los detalles de la receta: imagen, título, categoría, área, ingredientes con medidas, instrucciones, enlace a video. Obtiene ID de URL correctamente, agrega a recientes automáticamente, botón volver funciona	Muestra información principal con detalles menores faltantes	Muestra información pero con problemas significativos o no agrega a recientes	No muestra detalles o no funciona

Puntos clave a evaluar:

- ✅ Uso de `useParams()` para obtener ID
- ✅ Carga de datos de receta específica desde la API
- ✅ Muestra imagen, título, categoría, área
- ✅ Muestra lista de ingredientes con sus medidas
- ✅ Muestra instrucciones de preparación
- ✅ Muestra enlace a video de YouTube (si está disponible)
- ✅ Agrega automáticamente a recetas recientes usando contexto
- ✅ Botón "Volver al catálogo" funcional
- ✅ Manejo de receta no encontrada

---

9. Página 404 - NotFound (5 puntos)

Criterio	Excelente (5)	Bueno (4)	Regular (2-3)	Insuficiente (0-1)
Componente NotFound	Página 404 completa con emoji de cocina, mensaje de error 404, descripción amigable temática, botón de retorno funcional, estética consistente	Página 404 funcional con elementos menores faltantes	Página 404 básica sin estética o funcionalidad completa	No implementa página 404

Puntos clave a evaluar:

- ✅ Emoji de cocina visible (🍳)
- ✅ Mensaje "404 - Página no encontrada"
- ✅ Descripción amigable del error temática (relacionada con cocina/recetas)
- ✅ Botón "Volver al inicio" con `useNavigate()`
- ✅ Estética consistente con el resto de la aplicación

---

10. Comunicación con API (10 puntos)

Criterio	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (0-3)
Uso de API	Todas las peticiones se realizan correctamente a la API. Manejo adecuado de respuestas, errores de red manejados correctamente, estados de carga implementados	Peticiones funcionan con errores menores en manejo de errores o estados	Peticiones implementadas pero con problemas significativos	No se comunica con la API o no funciona

Puntos clave a evaluar:

- ✅ Peticiones a `http://localhost:3000/api/recetas`
- ✅ Peticiones a `http://localhost:3000/api/recetas/:id`
- ✅ Peticiones a `http://localhost:3000/api/categorias`
- ✅ Peticiones a `http://localhost:3000/api/areas`

- ✅ Manejo correcto de respuestas JSON
- ✅ Manejo de errores de red
- ✅ Manejo de errores de servidor (404, 500, etc.)

---

11. Modularidad y Organización del Código (5 puntos)

Criterio	Excelente (5)	Bueno (4)	Regular (2-3)	Insuficiente (0-1)
Código limpio y organizado	Componentes bien separados, código reutilizable, nombres descriptivos, comentarios cuando necesario, sin código repetido. Estructura de carpetas clara (components, pages, hooks, context, services)	Código organizado con mejoras menores posibles	Código funcional pero desorganizado o repetitivo	Código desorganizado y difícil de mantener

Puntos clave a evaluar:

- ✅ Componentes en archivos separados
- ✅ Servicios de API separados (carpeta `services/`)
- ✅ Nombres descriptivos de variables y funciones
- ✅ No hay código duplicado
- ✅ Imports organizados
- ✅ Código legible y mantenible
- ✅ Estructura de carpetas coherente

Criterios de Desaprobación Automática

El proyecto será desaprobado automáticamente si:

- ❌ La aplicación no ejecuta o tiene errores críticos que impiden su funcionamiento
- ❌ No se implementa el sistema de recetas recientes con Context API
- ❌ No se comunica con la API del servidor (usa archivos JSON estáticos en lugar de la API)

Bonus (Hasta 10 puntos adicionales)

Puntos extras por implementaciones adicionales:

- **+4 puntos**: Limpiar historial de recetas recientes (botón para eliminar todas)
- **+6 puntos**: Mostrar recetas similares en la página de detalles (misma categoría o área)

Notas Adicionales

Servidor API

IMPORTANTE: El servidor debe estar corriendo antes de iniciar la aplicación React. Si el servidor no está disponible, la aplicación debe manejar el error de manera elegante mostrando un mensaje al usuario.

Datos Importantes

- Cada receta incluye: nombre, categoría, área, imagen, ingredientes con medidas, instrucciones, y enlace a video (opcional)
- Las recetas pueden filtrarse por categoría y área
- Las recetas pueden buscarse por nombre
- Las recetas recientes se limitan a 15 y deben persistir en `localStorage`

Recomendaciones

1. Comienza configurando el routing básico
2. Implementa la carga de datos desde la API antes de diseñar componentes complejos
3. Crea el `RecetasRecientesContext` temprano para tener el estado global disponible
4. Usa componentes reutilizables para las cards de recetas
5. Implementa el debounce para la búsqueda para evitar demasiadas peticiones
6. Mantén la consistencia visual y la funcionalidad responsive

7. No olvides el manejo de errores y estados de carga
8. Asegúrate de que el servidor API esté corriendo durante el desarrollo
9. Prueba la persistencia de recetas recientes cerrando y abriendo la aplicación

Instrucciones para Ejecutar

1. **Iniciar el servidor API**:

```
cd server
npm install  # Solo la primera vez
node mockServer.js
```

2. **En otra terminal, iniciar la aplicación React**:

```
npm install
npm run dev
```

3. La aplicación estará disponible en `http://localhost:5173` (o el puerto que Vite asigne)